Module-5: Swings, Exploring Swing and Case Studies

Introducing Swing

Swing is a set of classes that provides more powerful and flexible GUI components than the earlier Abstract Window Toolkit (AWT). Swing provides buttons, lists, menus, and text areas that are platform-independent.

The Origins of Swing

Swing was developed to address the shortcomings of the AWT. AWT components rely on native code counterparts (peers), which leads to platform-specific limitations and the "lowest common denominator" feature set. Swing was introduced with the Java Foundation Classes (JFC) to provide a better alternative.

Swing Is Built on the AWT

Swing is built on top of the AWT, not a complete replacement. It reuses the core infrastructure of the AWT, including its event model and basic features, but replaces the AWT's component set with its own.

Two Key Swing Features

1. **Lightweight Components:** Swing components are "lightweight," meaning they are written entirely in Java and do not rely on native peers. This makes them more efficient, flexible, and consistent across different platforms.
2. **Pluggable Look and Feel (PLAF):** Swing separates the logic of a component from its appearance. This allows the look and feel of a Swing application to be changed dynamically at runtime. Several look and feels are available, such as Metal (the Java default), Windows, and Motif.

The MVC Connection

Swing components use a modified version of the Model-View-Controller (MVC) architecture.

- **Model:** Stores the state of the component.
- **View:** Determines how the component is displayed.
- **Controller:** Dictates how the component reacts to user input.

Swing combines the view and controller into a single logical entity called the **UI delegate**. This approach is often called Model-Delegate or Separable Model architecture.

Components and Containers

- **Component:** An object that has a graphical representation. JComponent is the top-level class for most Swing components.
- **Container:** A component that can hold other components. The Container class (from AWT) is the superclass for Swing containers.

Swing Containers:

1. **Top-Level Containers:** Do not inherit from JComponent. Every Swing GUI must have a top-level container.
    - JFrame: A typical window.
    - JApplet: An applet container.
    - JDialog: For creating dialog boxes.
2. **General-Purpose Containers:** Inherit from JComponent.
    - JPanel: A general-purpose container for grouping components.

The Swing Packages

Swing's classes are located in the javax.swing package and its subpackages, such as javax.swing.event, javax.swing.tree, and javax.swing.table.

A Simple Swing Application

All Swing applications are created on the **event dispatching thread**. The main thread should use SwingUtilities.invokeLater() to queue the GUI creation code for execution on this thread.

Example: A basic Swing application using JFrame and JLabel.

```
// A simple Swing application.

import javax.swing.*;



class SwingDemo {
```

```
    SwingDemo() {
        // Create a new JFrame container.
        JFrame jfrm = new JFrame("A Simple Swing Application");

        // Give the frame an initial size.
        jfrm.setSize(275, 100);

        // Terminate the program when the user closes the application.
        jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Create a text-based label.
        JLabel jlab = new JLabel(" Swing means powerful GUIs.");

        // Add the label to the content pane.
        jfrm.add(jlab);

        // Display the frame.
        jfrm.setVisible(true);
    }

    public static void main(String[] args) {
        // Create the frame on the event dispatching thread.
```

```
        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                new SwingDemo();
            }
        });
    }
}
```

7. Exploring Swing

This section covers several of the most commonly used Swing components.

JLabel and ImageIcon

JLabel is a component for placing text in a container. It can also display an icon.

Constructors:

- JLabel(Icon icon)
- JLabel(String str)
- JLabel(String str, Icon icon, int horizontalAlignment)

Example:

```
import java.awt.*;
import javax.swing.*;

class JLabelDemo {
    public JLabelDemo() {
        JFrame jfrm = new JFrame("JLabelDemo");
```

```
        jfrm.setLayout(new FlowLayout());
        jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        jfrm.setSize(260, 210);

        // Create an icon.
        ImageIcon ii = new ImageIcon("hourglass.png");

        // Create a label.
        JLabel jl = new JLabel("Hourglass", ii, JLabel.CENTER);

        // Add the label to the content pane.
        jfrm.add(jl);

        // Display the frame.
        jfrm.setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new JLabelDemo());
    }
}
```

JTextField

The JTextField class implements a single-line text-entry area.

Constructors:

- JTextField(int numChars)
- JTextField(String str)
- JTextField(String str, int numChars)

Example:

```
import java.awt.*;

import java.awt.event.*;

import javax.swing.*;


class JTextFieldDemo {

   public JTextFieldDemo() {

      JFrame jfrm = new JFrame("JTextFieldDemo");

      jfrm.setLayout(new FlowLayout());

      jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

      jfrm.setSize(260, 120);


      JTextField jtf = new JTextField(15);

      jfrm.add(jtf);


      JLabel jlab = new JLabel();

      jfrm.add(jlab);
```

```
        // Add an action listener for the text field.
        jtf.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent ae) {
                // Get text from the text field.
                jlab.setText(jtf.getText());
            }
        });

        jfrm.setVisible(true);
    }
    // ... main method
}
```

The Swing Buttons

Swing provides four types of buttons:

- JButton: A standard push button.
- JToggleButton: A toggled button with on/off states.
- JCheckBox: A check box.
- JRadioButton: A radio button (mutually exclusive).

JButton Example:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
class JButtonDemo implements ActionListener {
   JLabel jlab;

   public JButtonDemo() {
      // ... frame setup ...
      jfrm.setSize(220, 90);

      JButton jbtnUp = new JButton("Up");
      JButton jbtnDown = new JButton("Down");

      // Add action listeners.
      jbtnUp.addActionListener(this);
      jbtnDown.addActionListener(this);

      jfrm.add(jbtnUp);
      jfrm.add(jbtnDown);

      jlab = new JLabel("Press a button.");
      jfrm.add(jlab);

      jfrm.setVisible(true);
```

```
    }

    // Handle button events.
    public void actionPerformed(ActionEvent ae) {
        if(ae.getActionCommand().equals("Up")) {
            jlab.setText("You pressed Up.");
        } else {
            jlab.setText("You pressed Down.");
        }
    }
    // ... main method
}
```

A JToggleButton is a two-state button that stays in the "on" (pressed) or "off" (released) position until clicked again. Unlike a standard button that performs a single action, the toggle button maintains a state. It is often used for settings that can be switched back and forth, such as a "Mute" or "Bold" option.

/TOGGLE BUTTON EXAMPLE

```
// class myswing1 {

//     public static void main(String[] args) {

//         JFrame frame = new JFrame("Text Field Example");
//         frame.setLayout(new FlowLayout());
```

```
//        frame.setSize(300, 150);
//        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
//        JLabel jlab = new JLabel("Button is off.");
//        JToggleButton jtbn = new JToggleButton("On/Off");
//        frame.add(jlab);
//        frame.add(jtbn);

//        jtbn.addItemListener(new ItemListener() {
//            public void itemStateChanged(ItemEvent ie) {
//                if (jtbn.isSelected())
//                    jlab.setText("Button is on.");
//                else
//                    jlab.setText("Button is off.");
//            }
//        });

//        frame.setVisible(true);
//    }
// }
```

The JCheckBox allows a user to select one or more options from a set. Each checkbox operates independently, meaning checking one does not automatically uncheck another. In your example, ItemListener is used to detect when the box is checked or unchecked, allowing the program to respond to the user's specific selections.

```
//CHECK BOX EXAMPLE

// class myswing1 {



//     static JLabel jlab;



//     public static void main(String[] args) {

//         JFrame frame = new JFrame("Text Field Example");

//         JCheckBox checkBox1 = new JCheckBox("C++");

//         checkBox1.setBounds(300,300, 100,100);

//         JCheckBox checkBox2 = new JCheckBox("Java");

//         checkBox2.setBounds(350,250, 100,100);

//         frame.add(checkBox1);

//         frame.add(checkBox2);

//          frame.setLayout(new FlowLayout());

//          frame.setSize(400,400);

//          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

//          jlab = new JLabel("Select Language");

//          frame.add(jlab);


//         checkBox1.addItemListener(new ItemListener() {

//             public void itemStateChanged(ItemEvent e) {

//              jlab.setText("C++ Checkbox: "
```

```
//              + (e.getStateChange()==1?"checked":"unchecked"));
//          }
//        });
//        frame.setVisible(true);
//        }
//    }
```

A JRadioButton is used when the user must select exactly one option from a group. When several radio buttons are added to a ButtonGroup, selecting a new one will automatically deselect the previous one. This is ideal for mutually exclusive choices, such as selecting a gender or a single payment method

```
//RADIO BUTTON EXAMPLE
//  class myswing1 {


//     static JLabel jlab;
// public static void main(String[] args) {
// JFrame frame = new JFrame("Text Field Example");
//  JRadioButton r1=new JRadioButton("A) Male");
//        JRadioButton r2=new JRadioButton("B) Female");
//        r1.setBounds(75,50,100,30);
//        r2.setBounds(75,100,100,30);
 //        ButtonGroup bg=new ButtonGroup();
//        bg.add(r1);
//       bg.add(r2);
//       frame.add(r1);
```

```
//      frame.add(r2);
//      bg.addItemListener(new ItemListener() {
//          public void itemStateChanged(ItemEvent e) {
//           jlab.setText("selected: "
//             + (e.getStateChange()==1?"selected":"unselected"));
//          }
//        });
// }}
```

JTabbedPane

A JTabbedPane manages a set of components by linking them with tabs. Clicking a tab brings its associated component to the front.

Example:

```
// Demonstrate JTabbedPane.
import javax.swing.*;
public class JTabbedPaneDemo {
   public JTabbedPaneDemo() {
      JFrame jfrm = new JFrame("JTabbedPaneDemo");
      jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      jfrm.setSize(400, 200);


      JTabbedPane jtp = new JTabbedPane();
      jtp.addTab("Cities", new CitiesPanel());
      jtp.addTab("Colors", new ColorsPanel());
```

```
        jtp.addTab("Flavors", new FlavorsPanel());
        jfrm.add(jtp);

        jfrm.setVisible(true);
    }
    // ... main method and panel classes (e.g., CitiesPanel extends JPanel)
}
```

JScrollPane

A JScrollPane provides a scrollable view of a component.

Example:

```
import java.awt.*;
import javax.swing.*;

class JScrollPaneDemo {
    public JScrollPaneDemo() {
        JFrame jfrm = new JFrame("JScrollPaneDemo");
        jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        jfrm.setSize(400, 400);

        // Add 400 buttons to a panel.
        JPanel jp = new JPanel();
```

```
        jp.setLayout(new GridLayout(20, 20));
        for(int i = 0; i < 400; i++) {
            jp.add(new JButton("Button " + i));
        }

        // Create the scroll pane.
        JScrollPane jsp = new JScrollPane(jp);

        // Add the scroll pane to the content pane.
        jfrm.add(jsp, BorderLayout.CENTER);

        jfrm.setVisible(true);
    }
    // ... main method
}
```

JList

A JList provides a scrollable list of items from which the user can select one or more entries.

Example:

```
import javax.swing.*;
import javax.swing.event.*;
import java.awt.*;
```

```
class JListDemo implements ListSelectionListener {
   JList<String> jlst;
   JLabel jlab;
   String[] cities = { "New York", "Chicago", "Houston", "Denver",
                       "Los Angeles", "Seattle", "London", "Paris" };

   public JListDemo() {
      // ... frame setup ...
      jlst = new JList<>(cities);

      // Set list to single-selection mode.
      jlst.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

      JScrollPane jscrlp = new JScrollPane(jlst);
      jscrlp.setPreferredSize(new Dimension(120, 90));

      jlab = new JLabel("Please choose a city");

      // Add selection listener for the list.
      jlst.addListSelectionListener(this);
```

```
        jfrm.add(jscrlp);
        jfrm.add(jlab);
        jfrm.setVisible(true);
    }

    // Handle list selection events.
    public void valueChanged(ListSelectionEvent le) {
        int idx = jlst.getSelectedIndex();
        if (idx != -1)
            jlab.setText("Current selection: " + cities[idx]);
        else
            jlab.setText("Please choose a city");
    }
    // ... main method
}
```

JComboBox

A JComboBox is similar to a JList, but it displays a drop-down list.

Example:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
class JComboBoxDemo {
    String[] timepieces = { "Hourglass", "Analog", "Digital", "Stopwatch" };
    public JComboBoxDemo() {
        // ... frame setup ...
        JComboBox<String> jcb = new JComboBox<>(timepieces);
        jfrm.add(jcb);

        JLabel jlab = new JLabel(new ImageIcon("hourglass.png"));
        jfrm.add(jlab);

        jcb.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent ae) {
                String s = (String) jcb.getSelectedItem();
                jlab.setIcon(new ImageIcon(s.toLowerCase() + ".png"));
            }
        });
        jfrm.setVisible(true);
    }
    // ... main method
}
```

Trees

A JTree is a component that displays hierarchical data. Each node in the tree is a DefaultMutableTreeNode.

Example:

```
import java.awt.*;

import javax.swing.*;

import javax.swing.tree.*;


class JTreeDemo {

    public JTreeDemo() {

        // ... frame setup ...

        DefaultMutableTreeNode top = new DefaultMutableTreeNode("Options");


        DefaultMutableTreeNode a = new DefaultMutableTreeNode("A");

        top.add(a);

        a.add(new DefaultMutableTreeNode("A1"));

        a.add(new DefaultMutableTreeNode("A2"));


        DefaultMutableTreeNode b = new DefaultMutableTreeNode("B");

        top.add(b);

        b.add(new DefaultMutableTreeNode("B1"));

        b.add(new DefaultMutableTreeNode("B2"));
```

```
        JTree tree = new JTree(top);


        JScrollPane jsp = new JScrollPane(tree);
        jfrm.add(jsp);


        jfrm.setVisible(true);
    }
    // ... main method
}
```

JTable

A JTable displays data in a two-dimensional table.

Example:

```
import java.awt.*;
import javax.swing.*;


class JTableDemo {
    // Column headers.
    String[] colHeads = { "Name", "Extension", "ID#" };
    // Data.
    Object[][] data = {
        { "Gail", "4567", "865" },
```

```
        { "Ken", "7566", "555" },
        { "Viviane", "5634", "587" },
        // ... more data
    };

    public JTableDemo() {
        JFrame jfrm = new JFrame("JTableDemo");
        jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        jfrm.setSize(300, 200);

        JTable table = new JTable(data, colHeads);

        JScrollPane jsp = new JScrollPane(table);

        jfrm.add(jsp);
        jfrm.setVisible(true);
    }
    // ... main method
}
```

MODULE 3: BASIC STRUCTURE OF COMPUTERS

FUNCTIONAL UNITS

- A computer consists of 5 functionally independent main parts:
    1) Input
    2) Memory
    3) ALU
    4) Output &
    5) Control units.

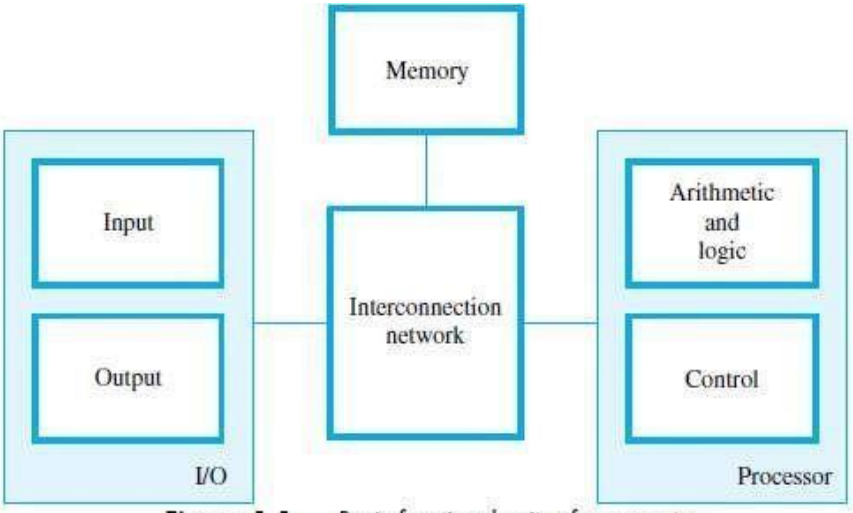


Figure 1.1 Basic functional units of a computer.

Input Unit : Computers accept coded information through input units. The most common input device is the keyboard. Whenever a key is pressed, the corresponding letter or digit is automatically translated into its corresponding binary code and transmitted to the processor.

Memory Unit: The function of the memory unit is to store programs and data. There are two classes of storage, called primary and secondary.
Primary Memory: Primary memory, also called main memory, is a fast memory that operates at electronic speeds. Programs must be stored in this memory while they are being executed.

Secondary Storage: Although primary memory is essential, it tends to be expensive and does not retain information when power is turned off. Thus additional, less expensive, permanent secondary storage is used when large amounts of data and many programs have to be stored, particularly for information that is accessed infrequently.

Output Unit : The output unit is the counterpart of the input unit. Its function is to send processed results to the outside world. Afamiliar example of such a device is a printer.

Control Unit: The memory, arithmetic and logic, and I/O units store and process information and perform input and output operations. The operation of these units must be coordinated in some way. This is the responsibility of the control unit.

BASIC OPERATIONAL CONCEPTS

- An Instruction consists of 2 parts, 1) Operation code (Opcode) and 2) Operands.

OPCODE	*OPERANDS*

- The data/operands are stored in memory.
- The individual instruction are brought from the memory to the processor.
- Then, the processor performs the specified operation.
- Let us see a typical instruction

  ADD LOCA, R0
- This instruction is an addition operation. The following are the steps to execute the instruction:

  Step 1: Fetch the instruction from main-memory into the processor.
  Step 2: Fetch the operand at location LOCA from main-memory into the processor.
  Step 3: Add the memory operand (i.e. fetched contents of LOCA) to the contents of register R0.
  Step 4: Store the result (sum) in R0.
- The same instruction can be realized using 2 instructions as:

  Load LOCA, R1
  Add R1, R0
- The following are the steps to execute the instruction:

  Step 1: Fetch the instruction from main-memory into the processor.
  Step 2: Fetch the operand at location LOCA from main-memory into the register R1.
  Step 3: Add the content of Register R1 and the contents of register R0.
  Step 4: Store the result (sum) in R0.
- The **processor** contains ALU, control-circuitry and many registers.
- The processor contains „n" general-purpose registers $\mathbf{R_0}$ through $\mathbf{R_{n-1}}$.
- The **IR** holds the instruction that is currently being executed.
- The **control-unit** generates the timing-signals that determine when a given action is to take place.
- The **PC** contains the memory-address of the next-instruction to be fetched & executed.
- During the execution of an instruction, the contents of PC are updated to point to next instruction.
- The **MAR** holds the address of the memory-location to be accessed.
- The **MDR** contains the data to be written into or read out of the addressed location.
- MAR and MDR facilitates the communication with memory.

  (IR → Instruction-Register, PC → Program Counter)
  (MAR → Memory Address Register, MDR→ Memory Data Register)

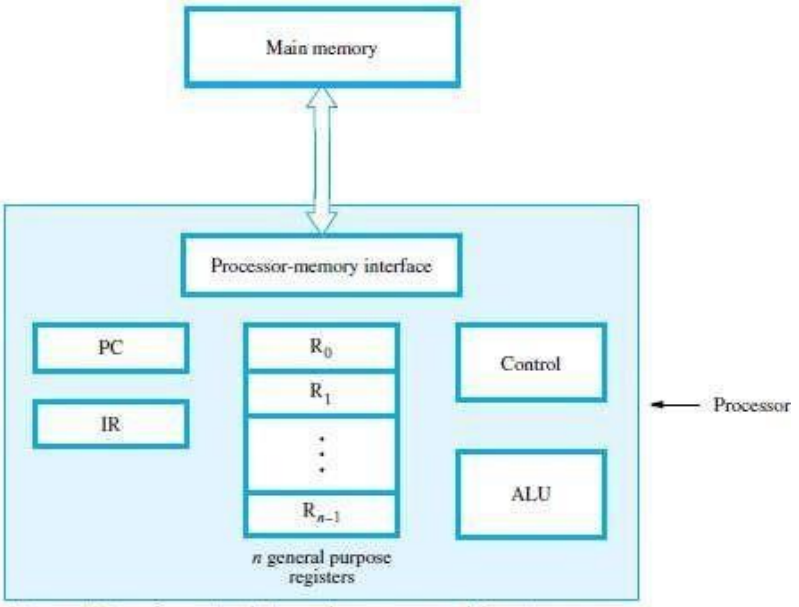


Figure 1.2 Connection between the processor and the main memory.

STEPS TO EXECUTE AN INSTRUCTION

1) The address of first instruction (to be executed) gets loaded into PC.
2) The contents of PC (i.e. address) are transferred to the MAR & control-unit issues Read signal to memory.
3) After certain amount of elapsed time, the first instruction is read out of memory and placed into MDR.
4) Next, the contents of MDR are transferred to IR. At this point, the instruction can be decoded & executed.
5) To fetch an operand, it's address is placed into MAR & control-unit issues Read signal. As a result, the operand is transferred from memory into MDR, and then it is transferred from MDR to ALU.
6) Likewise required number of operands is fetched into processor.
7) Finally, ALU performs the desired operation.
8) If the result of this operation is to be stored in the memory, then the result is sent to the MDR.
9) The address of the location where the result is to be stored is sent to the MAR and a Write cycle is initiated.
10) At some point during execution, contents of PC are incremented to point to next instruction in the program.

BUS STRUCTURE

- A bus is a group of lines that serves as a connecting path for several devices.
- A bus may be lines or wires.
- The lines carry data or address or control signal.
- There are 2 types of Bus structures: 1) Single Bus Structure and 2) Multiple Bus Structure.

1) Single Bus Structure

- Because the bus can be used for only one transfer at a time, only 2 units can actively use the bus at any given time.
- Bus control lines are used to arbitrate multiple requests for use of the bus.
- **Advantages:**
  1) Low cost &
  2) Flexibility for attaching peripheral devices.

2) Multiple Bus Structure

- Systems that contain multiple buses achieve more concurrency in operations.
- Two or more transfers can be carried out at the same time.
- **Advantage:** Better performance.
- **Disadvantage:** Increased cost.

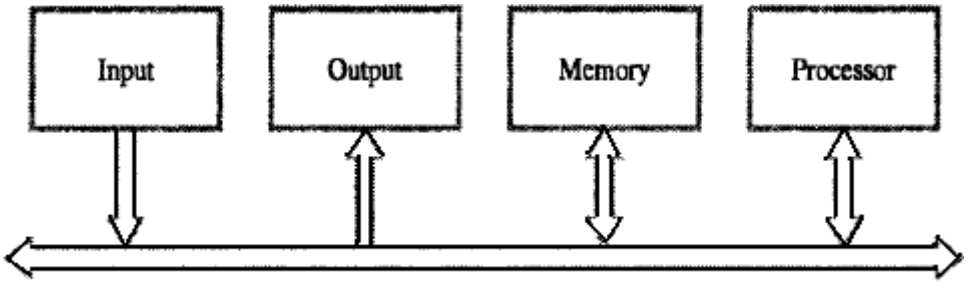


Figure 1.3 Single-bus structure.

- The devices connected to a bus vary widely in their speed of operation.
- To synchronize their operational-speed, buffer-registers can be used.

PERFORMANCE

- The most important measure of performance of a computer is how quickly it can execute programs.
- The speed of a computer is affected by the design of
  1) Instruction-set.
  2) Hardware & the technology in which the hardware is implemented.
  3) Software including the operating system.
- Because programs are usually written in a HLL, performance is also affected by the compiler that translates programs into machine language. (HLL→ High Level Language).
- For best performance, it is necessary to design the compiler, machine instruction set and hardware in a coordinated way.

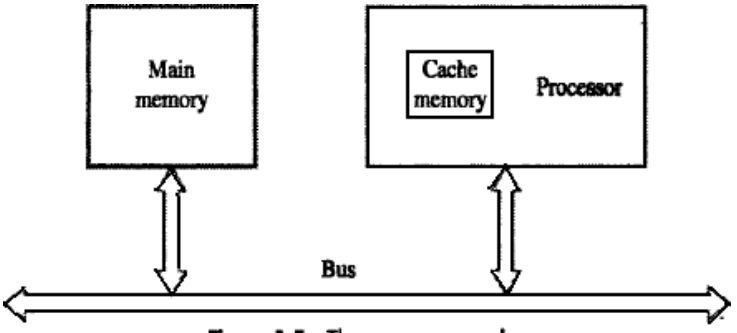


Figure 1.5 The processor cache.

- Let us examine the flow of program instructions and data between the memory & the processor.
- At the start of execution, all program instructions are stored in the main-memory.
- As execution proceeds, instructions are fetched into the processor, and a copy is placed in the cache.
- Later, if the same instruction is needed a second time, it is read directly from the cache.
- A program will be executed faster
  if movement of instruction/data between the main-memory and the processor is minimized
  which is achieved by using the cache.

PROCESSOR CLOCK

- Processor circuits are controlled by a timing signal called a **Clock**.
- The clock defines regular time intervals called **Clock Cycles**.
- To execute a machine instruction, the processor divides the action to be performed into a sequence of basic steps such that each step can be completed in one clock cycle.
- Let P = Length of one clock cycle
  R = Clock rate.
- Relation between P and R is given by

$$R = \frac{1}{P}$$

- R is measured in cycles per second.
- Cycles per second is also called Hertz (Hz)

BASIC PERFORMANCE EQUATION

- Let T = Processor time required to executed a program.
  N = Actual number of instruction executions.
  S = Average number of basic steps needed to execute one machine instruction.
  R = Clock rate in cycles per second.
- The program execution time is given by

$$T = \frac{N \times S}{R} \quad \text{.....(1)}$$

- Equ1 is referred to as the basic performance equation.
- To achieve high performance, the computer designer must reduce the value of T, which means reducing N and S, and increasing R.
  - The value of N is reduced if source program is compiled into fewer machine instructions.
  - The value of S is reduced if instructions have a smaller number of basic steps to perform.
  - The value of R can be increased by using a higher frequency clock.
- Care has to be taken while modifying values since changes in one parameter may affect the other.

CLOCK RATE

- There are 2 possibilities for increasing the clock rate R:
  1) Improving the IC technology makes logic-circuits faster.
     This reduces the time needed to compute a basic step. (IC → integrated circuits).
     This allows the clock period P to be reduced and the clock rate R to be increased.
  2) Reducing the amount of processing done in one basic step also reduces the clock period P.
- In presence of a cache, the percentage of accesses to the main-memory is small.
  Hence, much of performance-gain expected from the use of faster technology can be realized.
  The value of T will be reduced by same factor as R is increased „." S & N are not affected.

PERFORMANCE MEASUREMENT

- Benchmark refers to standard task used to measure how well a processor operates.
- The Performance Measure is the time taken by a computer to execute a given benchmark.
- SPEC selects & publishes the standard programs along with their test results for different application domains. (SPEC → System Performance Evaluation Corporation).
- SPEC Rating is given by

$$\text{SPEC rating} = \frac{\text{Running time on the reference computer}}{\text{Running time on the computer under test}}$$

- SPEC rating = 50 → The computer under test is 50 times as fast as reference-computer.
- The test is repeated for all the programs in the SPEC suite.
  Then, the geometric mean of the results is computed.
- Let $SPEC_i$ = Rating for program „i' in the suite.
  Overall SPEC rating for the computer is given by

$$\text{SPEC rating} = \left(\prod_{i=1}^{n} \text{SPEC}_i\right)^{\frac{1}{n}}$$

where n = no. of programs in the suite.

Solved Problems:

Problem 1:
List the steps needed to execute the machine instruction:
Load R2, LOC
in terms of transfers between the components of processor and some simple control commands.
Assume that the address of the memory-location containing this instruction is initially in register PC.
Solution:

1. Transfer the contents of register PC to register MAR.
2. Issue a Read command to memory.
   And, then wait until it has transferred the requested word into register MDR.
3. Transfer the instruction from MDR into IR and decode it.
4. Transfer the address LOCA from IR to MAR.
5. Issue a Read command and wait until MDR is loaded.
6. Transfer contents of MDR to the ALU.
7. Transfer contents of R0 to the ALU.
8. Perform addition of the two operands in the ALU and transfer result into R0.
9. Transfer contents of PC to ALU.
10. Add 1 to operand in ALU and transfer incremented address to PC.

Problem-2: List the steps needed to execute the machine instruction:
Add R4, R2, R3
in terms of transfers between the components of processor and some simple control commands.
Assume that the address of the memory-location containing this instruction is initially in register PC.
Solution:

1. Transfer the contents of register PC to register MAR.
2. Issue a Read command to memory.
   And, then wait until it has transferred the requested word into register MDR.
3. Transfer the instruction from MDR into IR and decode it.
4. Transfer contents of R1 and R2 to the ALU.
5. Perform addition of two operands in the ALU and transfer answer into R3.

6. Transfer contents of PC to ALU.
7. Add 1 to operand in ALU and transfer incremented address to PC.

Problem 4:

A program contains 1000 instructions. Out of that 25% instructions requires 4 clock cycles,40% instructions requires 5 clock cycles and remaining require 3 clock cycles for execution. Find the total time required to execute the program running in a 1 GHz machine.

Solution:

N = 1000

25% of N= 250 instructions require 4 clock cycles.

40% of N =400 instructions require 5 clock cycles.

35% of N=350 instructions require 3 clock cycles.

T = (N*S)/R= (250*4+400*5+350*3)/1X10^9 =(1000+2000+1050)/1*10^9= 4.05 μs.

Problem 5:

For the following processor, obtain the performance.

Clock rate = 800 MHz

No. of instructions executed = 1000

Average no of steps needed / machine instruction = 20

Solution:

$T = \frac{N \times S}{R}$ = (1000*20)/800 * 10^6=25 micro sec or 25*10^{-6} sec

MODULE 3 (CONT.): MACHINE INSTRUCTIONS &PROGRAMS

NUMBERS, ARITHMETIC OPERATIONS AND CHARACTERS

NUMBER REPRESENTATION

- Numbers can be represented in 3 formats:
  1) Sign and magnitude
  2) 1's complement
  3) 2's complement
- In all three formats, MSB=0 for +ve numbers & MSB=1 for -ve numbers.
- In **sign-and-magnitude system**,
  negative value is obtained by changing the MSB from 0 to 1 of the corresponding positive value.
  For ex, +5 is represented by 0101 &
  -5 is represented by 1101.
- In **1's complement system**,
  negative values are obtained by complementing each bit of the corresponding positive number.
  For ex, -5 is obtained by complementing each bit in 0101 to yield 1010.

(In other words, the operation of forming the 1's complement of a given number is equivalent to subtracting that number from 2^n-1).

- In **2's complement system**,
  forming the 2's complement of a number is done by subtracting that number from 2^n.
  For ex, -5 is obtained by complementing each bit in 0101 & then adding 1 to yield 1011.

(In other words, the 2's complement of a number is obtained by adding 1 to the 1's complement of that number).

- 2's complement system yields the most efficient way to carry out addition/subtraction operations.

B	Values represented		
$b_3b_2b_1b_0$	Sign and magnitude	1's complement	2's complement
0 1 1 1	+7	+7	+7
0 1 1 0	+6	+6	+6
0 1 0 1	+5	+5	+5
0 1 0 0	+4	+4	+4
0 0 1 1	+3	+3	+3
0 0 1 0	+2	+2	+2
0 0 0 1	+1	+1	+1
0 0 0 0	+0	+0	+0
1 0 0 0	−0	−7	−8
1 0 0 1	−1	−6	−7
1 0 1 0	−2	−5	−6
1 0 1 1	−3	−4	−5
1 1 0 0	−4	−3	−4
1 1 0 1	−5	−2	−3
1 1 1 0	−6	−1	−2
1 1 1 1	−7	−0	−1

Figure 1.3 Binary, signed-integer representations.

ADDITION OF POSITIVE NUMBERS

- Consider adding two 1-bit numbers.
- The sum of 1 & 1 requires the 2-bit vector 10 to represent the value 2. We say that sum is 0 and the carry-out is 1.

```
  0        1        0        1
+ 0      + 0      + 1      + 1
---      ---      ---      ---
  0        1        1       10
                             ↑
                          Carry-out
```

Figure 2.2 Addition of 1-bit numbers.

ADDITION & SUBTRACTION OF SIGNED NUMBERS

- Following are the two rules for addition and subtraction of n-bit signed numbers using the 2's complement representation system (Figure 1.6).

  Rule 1:

  - **To Add** two numbers, add their n-bits and ignore the carry-out signal from the MSB position.
  - Result will be algebraically correct, if it lies in the range -2^{n-1} to $+2^{n-1}-1$.

  Rule 2:

  - **To Subtract** two numbers X and Y (that is to perform X-Y), take the 2's complement of Y and then add it to X as in rule 1.
    - Result will be algebraically correct, if it lies in the range (2^{n-1}) to $+(2^{n-1}-1)$.
- When the result of an arithmetic operation is outside the representable-range, an arithmetic overflow is said to occur.
- To represent a signed in 2's complement form using a larger number of bits, repeat the sign bit as many times as needed to the left. This operation is called **sign extension**.
- In 1's complement representation, the result obtained after an addition operation is not always correct. The carry-out(c_n) cannot be ignored. If c_n=0, the result obtained is correct. If c_n=1, then a 1 must be added to the result to make it correct.

(a)	0010	(+2)	(b)	0100	(+4)
	+ 0011	(+3)		+ 1010	(–6)
	0101	(+5)		1110	(–2)
(c)	1011	(–5)	(d)	0111	(+7)
	+ 1110	(–2)		+ 1101	(–3)
	1001	(–7)		0100	(+4)
(e)	1101	(–3)	⇒	1101	
	– 1001	(–7)		+ 0111	
				0100	(+4)
(f)	0010	(+2)	⇒	0010	
	– 0100	(+4)		+ 1100	
				1110	(–2)
(g)	0110	(+6)	⇒	0110	
	– 0011	(+3)		+ 1101	
				0011	(+3)
(h)	1001	(–7)	⇒	1001	
	– 1011	(–5)		+ 0101	
				1110	(–2)
(i)	1001	(–7)	⇒	1001	
	– 0001	(+1)		+ 1111	
				1000	(–8)
(j)	0010	(+2)	⇒	0010	
	– 1101	(–3)		+ 0011	
				0101	(+5)

Figure 1.6 2's-complement Add and Subtract operations.

OVERFLOW IN INTEGER ARITHMETIC

- When result of an arithmetic operation is outside the representable-range, an **arithmetic overflow** is said to occur.
- For example: If we add two numbers +7 and +4, then the output sum S is 1011(←0111+0100), which is the code for -5, an incorrect result.
- An overflow occurs in following 2 cases

  1) Overflow can occur only when adding two numbers that have the same sign.
  2) The carry-out signal from the sign-bit position is not a sufficient indicator of overflow when adding signed numbers.

Solved Problems:

Problem 1:

Represent the decimal values 5, -2, 14, -10, 26, -19, 51 and -43 as signed 7-bit numbers in the following binary formats:

(a) sign-and-magnitude
(b) 1's-complement
(c) 2's-complement

Solution:

The three binary representations are given as:

Decimal values	Sign-and-magnitude representation	1's-complement representation	2's-complement representation
5	0000101	0000101	0000101
−2	1000010	1111101	1111110
14	0001110	0001110	0001110
−10	1001010	1110101	1110110
26	0011010	0011010	0011010
−19	1010011	1101100	1101101
51	0110011	0110011	0110011
−43	1101011	1010100	1010101

Problem 2:

(a) Convert the following pairs of decimal numbers to 5-bit 2's-complement numbers, then add them. State whether or not overflow occurs in each case.

a) 5 and 10 b) 7 and 13
c) −14 and 11 d) −5 and 7
e) −3 and −8

(b) Repeat Problem 1.7 for the subtract operation, where the second number of each pair is to be subtracted from the first number. State whether or not overflow occurs in each case.

Solution:

(a)

$$
\begin{array}{lrlrlr}
(a) & 00101 & (b) & 00111 & (c) & 10010 \\
 & +\ 01010 & & +\ 01101 & & +\ 01011 \\
 & \overline{01111} & & \overline{10100} & & \overline{11101} \\
 & \text{no overflow} & & \text{overflow} & & \text{no overflow} \\
(d) & 11011 & (e) & 11101 & (f) & 10110 \\
 & +\ 00111 & & +\ 11000 & & +\ 10011 \\
 & \overline{00010} & & \overline{10101} & & \overline{01001} \\
 & \text{no overflow} & & \text{no overflow} & & \text{overflow}
\end{array}
$$

(b) To subtract the second number, form its 2's-complement and add it to the first number.

$$
\begin{array}{lrlrlr}
(a) & 00101 & (b) & 00111 & (c) & 10010 \\
 & +\ 10110 & & +\ 10011 & & +\ 10101 \\
 & \overline{11011} & & \overline{11010} & & \overline{00111} \\
 & \text{no overflow} & & \text{no overflow} & & \text{overflow} \\
(d) & 11011 & (e) & 11101 & (f) & 10110 \\
 & +\ 11001 & & +\ 01000 & & +\ 01101 \\
 & \overline{10100} & & \overline{00101} & & \overline{00011} \\
 & \text{no overflow} & & \text{no overflow} & & \text{no overflow}
\end{array}
$$

Problem 3:
Perform following operations on the 6-bit signed numbers using 2's complement representation system. Also indicate whether overflow has occurred.

010110	101011	111111
+001001	+100101	+000111
011001	110111	010101
+010000	+111001	+101011
010110	111110	100001
−011111	−100101	−011101
111111	000111	011010
−000111	−111000	−100010

Solution:

010110	(+22)	101011	(−21)	111111	(−1)
+ 001001	+ (+9)	+ 100101	+ (−27)	+ 000111	+ (+7)
011111	(+31)	010000	(−48)	000110	(+6)
		overflow			
011001	(+25)	110111	(−9)	010101	(+21)
+ 010000	+ (+16)	+ 111001	+ (−7)	+ 101011	+ (−21)
101001	(+41)	110000	(−16)	000000	(0)
overflow					

010110	(+22)	010110
− 011111	− (+31)	+ 100001
	(−9)	110111
111110	(−2)	111110
− 100101	− (−27)	+ 011011
	(+25)	011001
100001	(−31)	100001
− 011101	− (+29)	+ 100011
	(−60)	000100
		overflow
111111	(−1)	111111
− 000111	− (+7)	+ 111001
	(−8)	111000
000111	(+7)	000111
− 111000	− (−8)	+ 001000
	(+15)	001111
011010	(+26)	011010
− 100010	− (−30)	+ 011110
	(+56)	111000
		overflow

Problem 4 : Consider a system that uses 5 bits for representing signed integers in 2's complement format. In this system, two integers A and B are represented as A=01010 and B=11010. Which one of the following operations will result in either an arithmetic overflow?

i) A+B

ii)A-B

iii)B-A

Iv) 2xB

Solution :Option ii) A-B

Explanation : First, let's understand the representation of the integers A and B in 2's complement format. The given 5-bit binary numbers need to be interpreted as follows:

For A (01010):

The first bit (0) indicates that the number is positive. Hence, we directly convert the remaining bits (1010) to decimal.

For B (11010):

The first bit (1) indicates that the number is negative. To find the decimal value, we first find the 2's complement.

Step 1: Invert all bits: 00101

Step 2: Add 1 to the least significant bit (LSB): 00110

This gives us the positive equivalent of the negative number. Converting 00110 to decimal:

Since the original number was negative, B = -6.

Now we have:

A = 10

B = -6

Let's perform the given operations and check for overflow or underflow:

Option A: A + B

The result 4 can be represented in 5 bits: 00100.

No overflow or underflow here.

Option B: A - B

The result 16 cannot be represented in 5 bits signed integers, which can range from -16 to 15:

5-bit binary for 16 would need more than 5 bits: 10000.

Hence, this operation results in an arithmetic overflow.

Option C: B - A

-16 can be represented in 5 bits: 10000.

There is no overflow or underflow in this case, as -16 is within the representable range.

Option D: 2 * B

-12 can be represented in 5 bits: 10100.

No overflow or underflow here.

MODULE 3 (CONT.): MEMORY-LOCATIONS & ADDRESSES

- **Memory** consists of many millions of storage cells (flip-flops).
- Each cell can store a bit of information i.e. 0 or 1 (Figure 2.1).
- Each group of n bits is referred to as a **word** of information, and n is called the **word length**.
- The word length can vary from 8 to 64 bits.
- A unit of 8 bits is called a **byte**.
- Accessing the memory to store or retrieve a single item of information (word/byte) requires distinct addresses for each item location. (It is customary to use numbers from 0 through 2^k-1 as the addresses of successive-locations in the memory).
- If 2^k = no. of addressable locations;

  then 2^k addresses constitute the address-space of the computer.

  For example, a 24-bit address generates an address-space of 2^{24} locations (16 MB).

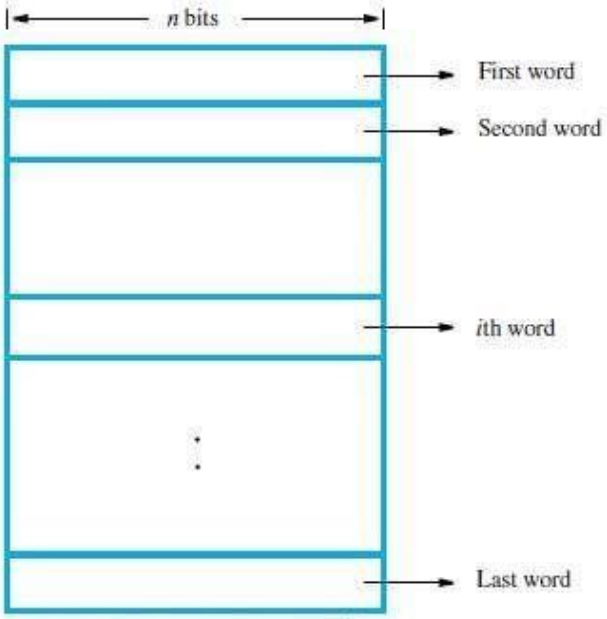


Figure 2.1 Memory words.

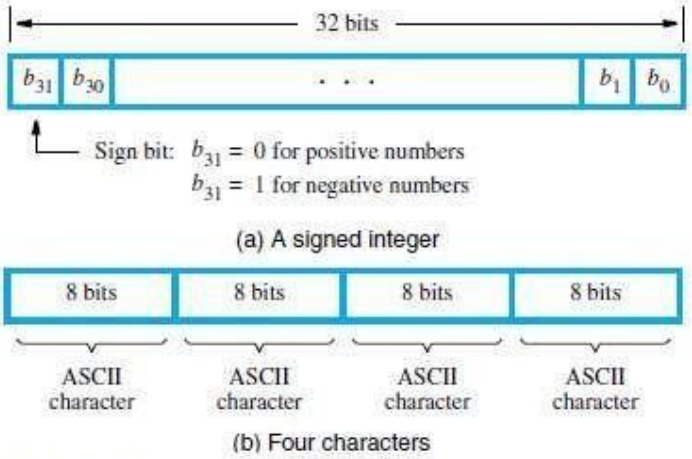


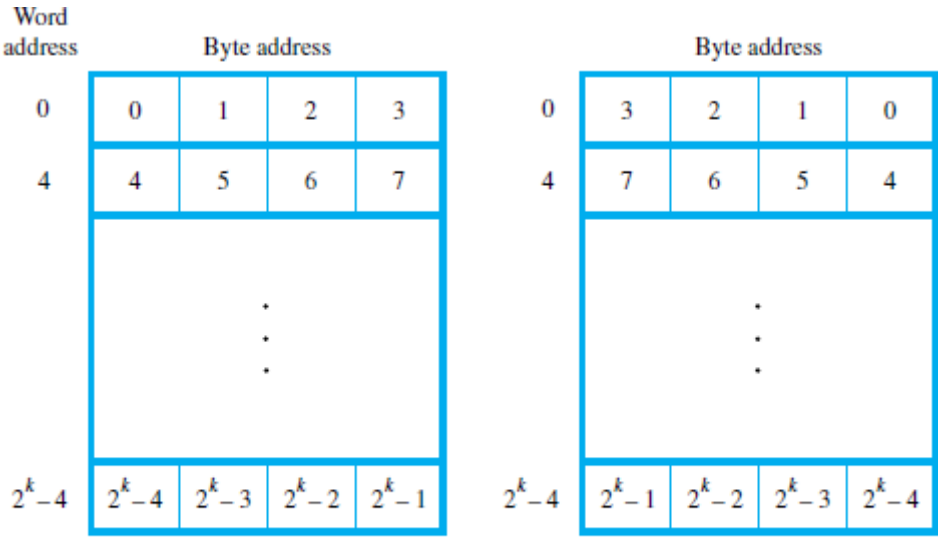
Figure 2.2 Examples of encoded information in a 32-bit word.

BYTE-ADDRESSABILITY

- In byte-addressable memory, successive addresses refer to successive byte locations in the memory.
- Byte locations have addresses 0, 1, 2. . . . .
- If the word-length is 32 bits, successive words are located at addresses 0, 4, 8. . with each word having 4 bytes.

BIG-ENDIAN & LITTLE-ENDIAN ASSIGNMENTS

- There are two ways in which byte-addresses are arranged (Figure 2.3).
  1) **Big-Endian:** Lower byte-addresses are used for the more significant bytes of the word.
  2) **Little-Endian:** Lower byte-addresses are used for the less significant bytes of the word
- In both cases, byte-addresses 0, 4, 8.....are taken as the addresses of successive words in the memory.



(a) Big-endian assignment (b) Little-endian assignment

Figure 2.3 Byte and word addressing.

- Consider a 32-bit integer (in hex): 0**x**12345678 which consists of 4 bytes: 12, 34, 56, and 78.
  - Hence this integer will occupy 4 bytes in memory.
  - Assume, we store it at memory address starting 1000.
  - On little-endian, memory will look like

Address	Value
1000	78
1001	56
1002	34
1003	12

  - On big-endian, memory will look like

Address	Value
1000	12
1001	34
1002	56
1003	78

WORD ALIGNMENT

- Words are said to be **Aligned** in memory if they begin at a byte-address that is a multiple of the number of bytes in a word.
- For example,
  - If the word length is 16(2 bytes), aligned words begin at byte-addresses 0, 2, 4 . . . . .
  - If the word length is 64(2 bytes), aligned words begin at byte-addresses 0, 8, 16 . . . . .
- Words are said to have **Unaligned Addresses**, if they begin at an arbitrary byte-address.

ACCESSING NUMBERS, CHARACTERS & CHARACTERS STRINGS

- A number usually occupies one word. It can be accessed in the memory by specifying its word address. Similarly, individual characters can be accessed by their byte-address.
- There are two ways to indicate the length of the string:
  1) A special control character with the meaning "end of string" can be used as the last character in the string.
  2) A separate memory word location or register can contain a number indicating the length of the string in bytes.

Solved Problems:

Problem 1: A memory Byte location contains the pattern 00101100. What does this pattern represent when interpreted as a binary number ? What does it represent as an ASCII code ?

Solution: The number 44 and the ASCII punctuation character " , "

Problem 2:
Consider a Computer that has a byte addressable memory organized in 32 bit words according to the **Bi-endian scheme**. A program treads ASCII characters entered at a keyboard and store them in successive byte locations, starting at location 1000. Show the content of the two memory words at locations 1000 and 1004 After the name " Johnson" has been entered.

Solution : Byte contents in hex, starting at location 1000, will be 4A, 6F, 68, 6E, 73, 6F, 6E. The two words at 1000 and 1004 will be 4A6F686E and 736F6EXX. Byte 1007 (shown as XX) is unchanged.

(ASCII Codes : J=4A, o=6F, h=68, n=6E, s=73).

Problem 3:
Consider a Computer that has a byte addressable memory organized in 32 bit words according to the little-**endian scheme**. A program treads ASCII characters entered at a keyboard and store them in successive byte locations, starting at location 1000. Show the content of the two memory words at locations 1000 and 1004 After the name " Johnson" has been entered.

Solution: Byte contents in hex, starting at location 1000, will be 4A, 6F, 68, 6E, 73, 6F, 6E. The two words at 1000 and 1004 will be 6E686F4A and XX6E6F73. Byte 1007 (shown as XX) is unchanged.

-MEMORY OPERATIONS

- Two memory operations are:
  1) Load (Read/Fetch) &
  2) Store (Write).
- The **Load** operation transfers a copy of the contents of a specific memory-location to the processor. The memory contents remain unchanged.
- Steps for Load operation:
  1) Processor sends the address of the desired location to the memory.
  2) Processor issues „read" signal to memory to fetch the data.
  3) Memory reads the data stored at that address.
  4) Memory sends the read data to the processor.
- The **Store** operation transfers the information from the register to the specified memory-location. This will destroy the original contents of that memory-location.
- Steps for Store operation are:
  1) Processor sends the address of the memory-location where it wants to store data.
  2) Processor issues „write" signal to memory to store the data.
  3) Content of register(MDR) is written into the specified memory-location.

INSTRUCTIONS & INSTRUCTION SEQUENCING

- A computer must have instructions capable of performing 4 types of operations:
  1) Data transfers between the memory and the registers (MOV, PUSH, POP, XCHG).
  2) Arithmetic and logic operations on data (ADD, SUB, MUL, DIV, AND, OR, NOT).
  3) Program sequencing and control (CALL.RET, LOOP, INT).
  4) I/0 transfers (IN, OUT).

REGISTER TRANSFER NOTATION (RTN)

- The possible locations in which transfer of information occurs are: 1) Memory-location 2) Processor register & 3) Registers in I/O device.

Location	Hardware Binary Address	Example	Description
Memory	LOC, PLACE, NUM	R1 ← [LOC]	Contents of memory-location LOC are transferred into register R1.
Processor	R0, R1 ,R2	[R3] ← [R1]+[R2]	Add the contents of register R1 &R2 and places their sum into R3.
I/O Registers	DATAIN, DATAOUT	R1 ← DATAIN	Contents of I/O register DATAIN are transferred into register R1.

ASSEMBLY LANGUAGE NOTATION

- To represent machine instructions and programs, assembly language format is used.

Assembly Language Format	Description
Move LOC, R1	Transfer data from memory-location LOC to register R1. The contents of LOC are unchanged by the execution of this instruction, but the old contents of register R1 are overwritten.
Add R1, R2, R3	Add the contents of registers R1 and R2, and places their sum into register R3.

BASIC INSTRUCTION TYPES

Instruction Type	Syntax	Example	Description	Instructions for Operation C<-[A]+[B]
Three Address	Opcode Source1,Source2,Destination	Add A,B,C	Add the contents of memory-locations A & B. Then, place the result into location C.	
Two Address	Opcode Source, Destination	Add A,B	Add the contents of memory-locations A & B. Then, place the result into location B, replacing the original contents of this location. Operand B is both a source and a destination.	Move B, C Add A, C
One Address	Opcode Source/Destination	Load A	Copy contents of memory-location A into accumulator.	Load A Add B Store C
		Add B	Add contents of memory-location B to contents of accumulator register & place sum back into accumulator.	
		Store C	Copy the contents of the accumulator into location C.	
Zero Address	Opcode [no Source/Destination]	Push	Locations of all operands are defined implicitly. The operands are stored in a pushdown stack.	Not possible

- Access to data in the registers is much faster than to data stored in memory-locations.
- Let Ri represent a general-purpose register.
-
- The instructions: *Load A,Ri*
  Store Ri,A

Add A,Ri

are generalizations of the Load, Store and Add Instructions for the single-accumulator case, in which register Ri performs the function of the accumulator.

• In processors, where arithmetic operations as allowed only on operands that are in registers, the task C<-[A]+[B] can be performed by the instruction sequence:

Move A,Ri
Move B,Rj
Add Ri,Rj
Move Rj,C

Solved Problems:

Problem1 :Write a program that can evaluate the expression A*B+C*D In a single-accumulator processor.Assume that the processor has Load, Store, Multiply, and Add instructions and that all values fit in the accumulator

Solution:

A program for the expression is:

Load A
Multiply B
Store RESULT
Load C
Multiply D
Add RESULT
Store RESULT

INSTRUCTION EXECUTION & STRAIGHT LINE SEQUENCING

- The program is executed as follows:
  1) Initially, the address of the first instruction is loaded into PC (Figure 2.8).
  2) Then, the processor control circuits use the information in the PC to fetch and execute instructions, one at a time, in the order of increasing addresses. This is called *Straight-Line sequencing***.**
  3) During the execution of each instruction, PC is incremented by 4 to point to next instruction.
- There are 2 phases for Instruction Execution:
  1) Fetch Phase: The instruction is fetched from the memory-location and placed in the IR.
  2) Execute Phase: The contents of IR is examined to determine which operation is to be performed. The specified-operation is then performed by the processor.

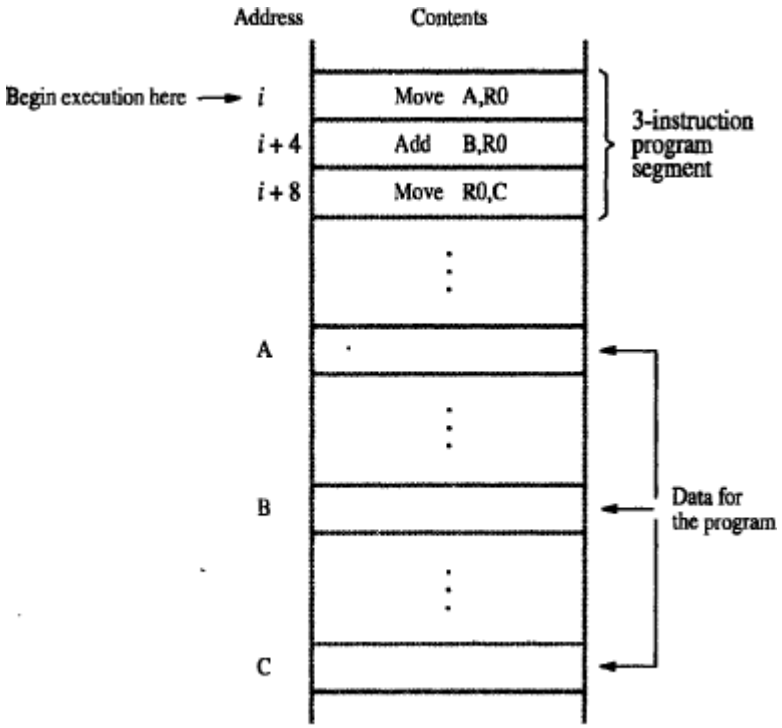


Figure 2.8 A program for C ← [A] + [B].

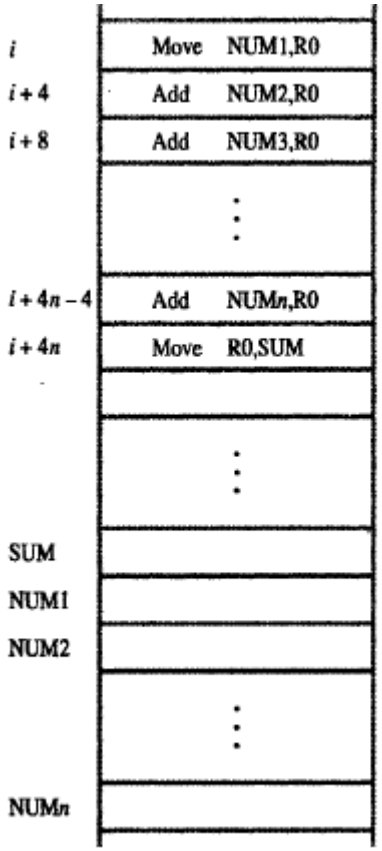


Figure 2.9 A straight-line program for adding *n* numbers.

Program Explanation

- Consider the program for adding a list of n numbers (Figure 2.9).
- The Address of the memory-locations containing the n numbers are symbolically given as NUM1, NUM2…..NUMn.
- Separate Add instruction is used to add each number to the contents of register R0.
- After all the numbers have been added, the result is placed in memory-location SUM.

BRANCHING

- Consider the task of adding a list of „n" numbers (Figure 2.10).
- Number of entries in the list „n" is stored in memory-location **N**.
- Register **R1** is used as a counter to determine the number of times the loop is executed.
- Content-location N is loaded into register R1 at the beginning of the program.
- The **Loop** is a straight line sequence of instructions executed as many times as needed.
  The loop starts at location LOOP and ends at the instruction Branch>0.
- During each pass,
  → address of the next list entry is determined and
  → that entry is fetched and added to R0.

• The instruction *Decrement R1* reduces the contents of R1 by 1 each time through the loop.
• Then **Branch Instruction** loads a new value into the program counter. As a result, the processor fetches and executes the instruction at this new address called the **Branch Target**.
• A **Conditional Branch Instruction** causes a branch only if a specified condition is satisfied. If the condition is not satisfied, the PC is incremented in the normal way, and the next instruction in sequential address order is fetched and executed.

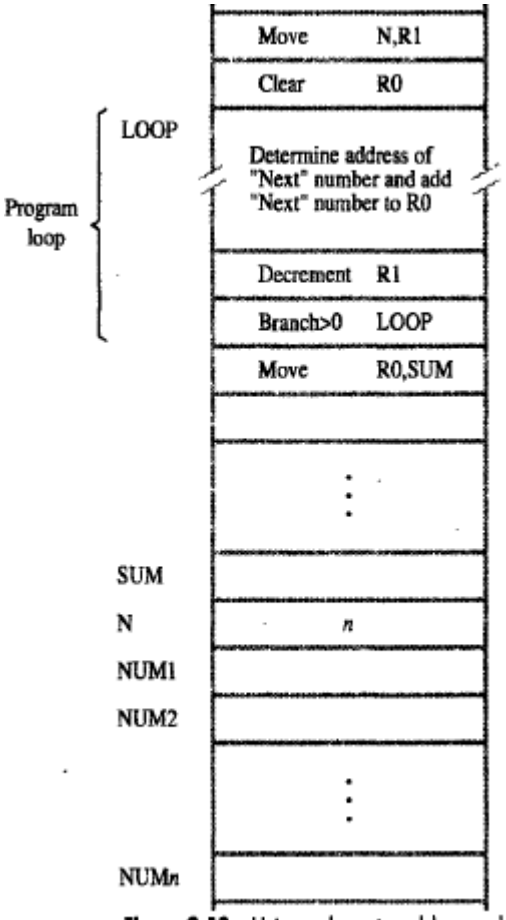


Figure 2.10 Using a loop to add *n* numbers.

CONDITION CODES

• The processor keeps track of information about the results of various operations. This is accomplished by recording the required information in individual bits, called **Condition Code Flags**.
• These flags are grouped together in a special processor-register called the condition code register (or statue register).
• Four commonly used flags are:

1) N (negative) set to 1 if the result is negative, otherwise cleared to 0.
2) Z (zero) set to 1 if the result is 0; otherwise, cleared to 0.
3) V (overflow) set to 1 if arithmetic overflow occurs; otherwise, cleared to 0.
4) C (carry) set to 1 if a carry-out results from the operation; otherwise cleared to 0.

MODULE 3 (CONT.): INTERRUPTS

- There are many situations where other tasks can be performed while waiting for an I/O device to become ready.
- A hardware signal called an Interrupt will alert the processor when an I/O device becomes ready.
- Interrupt-signal is sent on the interrupt-request line.
- The processor can be performing its own task without the need to continuously check the I/O-device.
- The routine executed in response to an interrupt-request is called ISR.
- The processor must inform the device that its request has been recognized by sending INTA signal.
  (INTR → Interrupt Request, INTA → Interrupt Acknowledge, ISR → Interrupt Service Routine)
- For example, consider COMPUTE and PRINT routines (Figure 3.6).

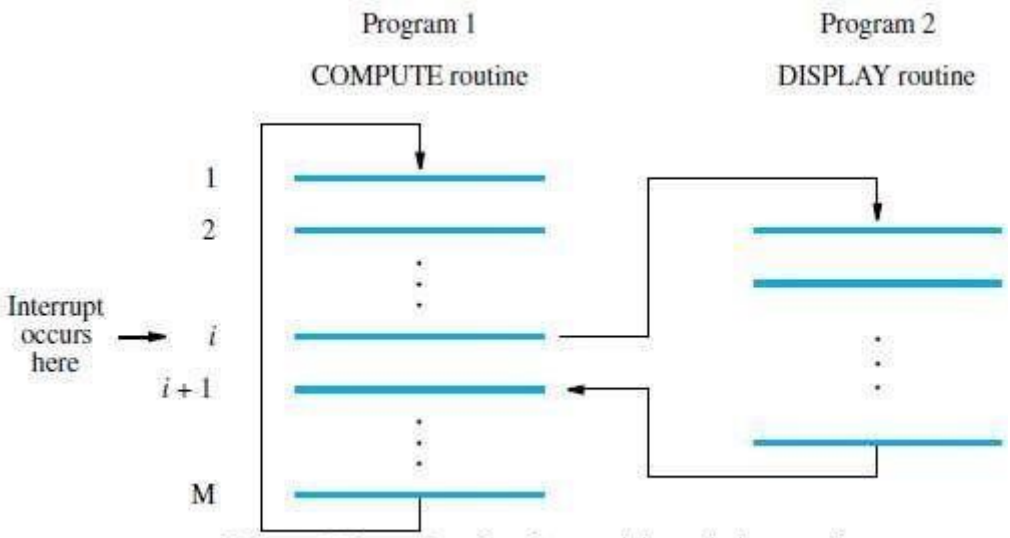


Figure 3.6 Transfer of control through the use of interrupts.

- The processor first completes the execution of instruction i.
- Then, processor loads the PC with the address of the first instruction of the ISR.
- After the execution of ISR, the processor has to come back to instruction i+1.
- Therefore, when an interrupt occurs, the current content of PC is put in temporary storage location.
- A return at the end of ISR reloads the PC from that temporary storage location.
- This causes the execution to resume at instruction i+1.
- When processor is handling interrupts, it must inform device that its request has been recognized.
- This may be accomplished by INTA signal.
- The task of saving and restoring the information can be done automatically by the processor.
- The processor saves only the contents of **PC & Status register.**
- Saving registers also increases the Interrupt Latency.
- **Interrupt Latency** is a delay between
  → time an interrupt-request is received and
  → start of the execution of the ISR.
- Generally, the long interrupt latency in unacceptable.

Difference between Subroutine & ISR

Subroutine	ISR
A subroutine performs a function required by the program from which it is called.	ISR may not have anything in common with program being executed at time INTR is received
Subroutine is just a linkage of 2 or more function related to each other.	Interrupt is a mechanism for coordinating I/O transfers.

INTERRUPT HARDWARE

- Most computers have several I/O devices that can request an interrupt.
- A single interrupt-request (IR) line may be used to serve n devices (Figure 4.6).
- All devices are connected to IR line via switches to ground.
- To request an interrupt, a device closes its associated switch.
- Thus, if all IR signals are inactive, the voltage on the IR line will be equal to V_{dd}.
- When a device requests an interrupt, the voltage on the line drops to 0.
- This causes the INTR received by the processor to go to 1.
- The value of INTR is the logical OR of the requests from individual devices.

  $$\mathbf{INTR = INTR_1 + INTR_2 + \ldots\ldots\ldots\ldots + INTR_n}$$

- A special gates known as open-collector or open-drain are used to drive the INTR line.
- The Output of the open collector control is equal to a switch to the ground that is
  - → open when gates input is in "0" state and
  - → closed when the gates input is in "1" state.
- Resistor R is called a **Pull-up Resistor** because
  it pulls the line voltage up to the high-voltage state when the switches are open.

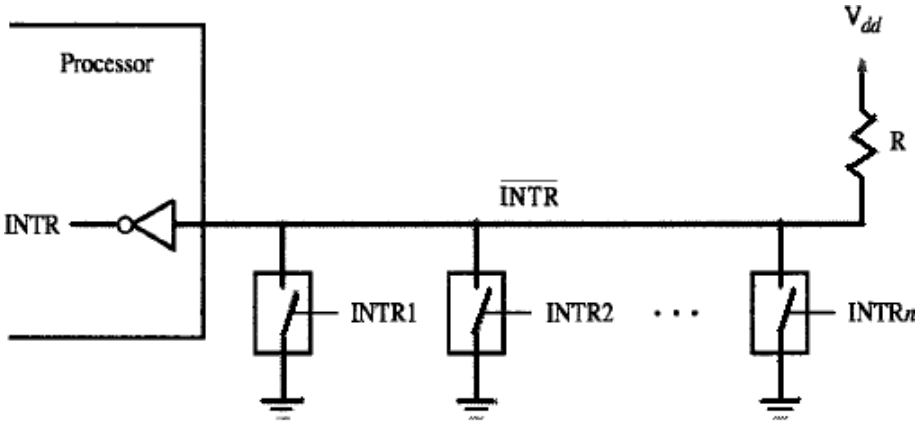


Figure 4.6 An equivalent circuit for an open-drain bus used to implement a common interrupt-request line.

ENABLING & DISABLING INTERRUPTS

- All computers fundamentally should be able to enable and disable interruptions as desired.
- The problem of infinite loop occurs due to successive interruptions of active INTR signals.
- There are 3 mechanisms to solve problem of infinite loop:
  1) Processor should ignore the interrupts until execution of first instruction of the ISR.
  2) Processor should automatically disable interrupts before starting the execution of the ISR.
  3) Processor has a special INTR line for which the interrupt-handling circuit.
     Interrupt-circuit responds only to leading edge of signal. Such line is called edge-triggered.
- Sequence of events involved in handling an interrupt-request:
  1) The device raises an interrupt-request.
  2) The processor interrupts the program currently being executed.
  3) Interrupts are disabled by changing the control bits in the processor status register (PS).
  4) The device is informed that its request has been recognized.
     In response, the device deactivates the interrupt-request signal.
  5) The action requested by the interrupt is performed by the interrupt-service routine.
  6) Interrupts are enabled and execution of the interrupted program is resumed.

HANDLING MULTIPLE DEVICES

- While handling multiple devices, the issues concerned are:
  1) How can the processor recognize the device requesting an interrupt?
  2) How can the processor obtain the starting address of the appropriate ISR?
  3) Should a device be allowed to interrupt the processor while another interrupt is being serviced?
  4) How should 2 or more simultaneous interrupt-requests be handled?

1)VECTORED INTERRUPTS

- A device requesting an interrupt identifies itself by sending a special-code to processor over bus.
- Then, the processor starts executing the ISR.
- The special-code indicates starting-address of ISR.
- The special-code length ranges from 4 to 8 bits.
- The location pointed to by the interrupting-device is used to store the staring address to ISR.
- The staring address to ISR is called the **interrupt vector**.
- Processor
  → loads interrupt-vector into PC &
  → executes appropriate ISR.
- When processor is ready to receive interrupt-vector code, it activates INTA line.
- Then, I/O-device responds by sending its interrupt-vector code & turning off the INTR signal.
- The interrupt vector also includes a new value for the Processor Status Register.

2)INTERRUPT NESTING

- A multiple-priority scheme is implemented by using separate INTR & INTA lines for each device
- Each INTR line is assigned a different priority-level (Figure 4.7).
- Priority-level of processor is the priority of program that is currently being executed.
- Processor accepts interrupts only from devices that have higher-priority than its own.
- At the time of execution of ISR for some device, priority of processor is raised to that of the device.
- Thus, interrupts from devices at the same level of priority or lower are disabled.

Privileged Instruction

- Processor's priority is encoded in a few bits of PS word. (PS → Processor-Status).
- Encoded-bits can be changed by **Privileged Instructions** that write into PS.
- Privileged-instructions can be executed only while processor is running in **Supervisor Mode**.
- Processor is in supervisor-mode only when executing operating-system routines.

Privileged Exception

- User program cannot
  → accidently or intentionally change the priority of the processor &
  → disrupt the system-operation.
- An attempt to execute a privileged-instruction while in user-mode leads to a **Privileged Exception**.

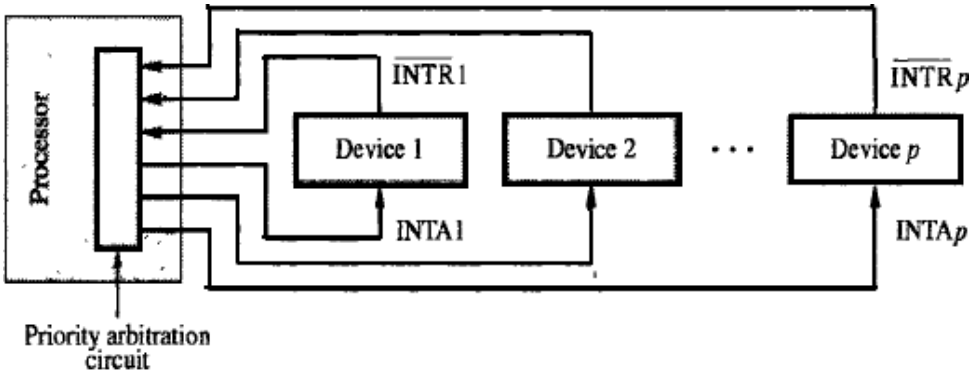


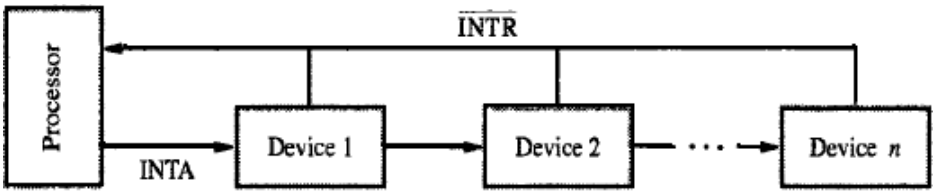
Figure 4.7 Implementation of interrupt priority using individual interrupt-request and acknowledge lines.

3)SIMULTANEOUS REQUESTS

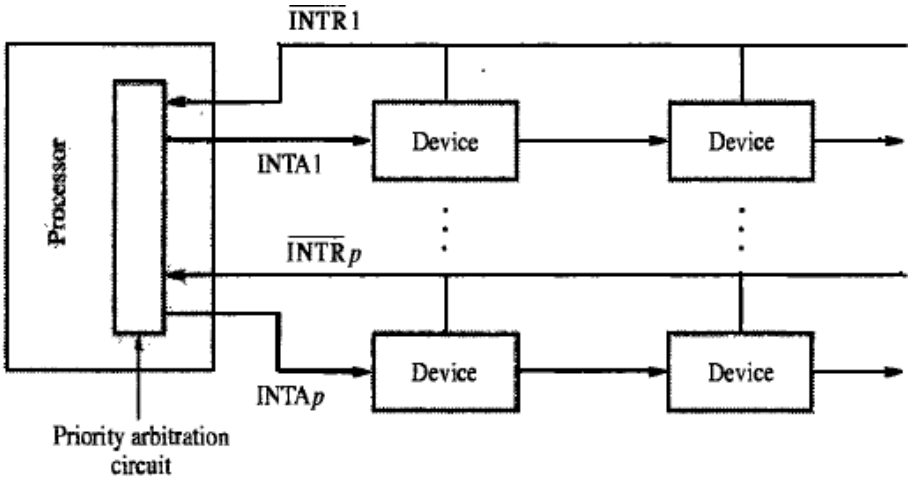
• The processor must have some mechanisms to decide which request to service when simultaneous requests arrive.
• INTR line is common to all devices (Figure 4.8a).
• INTA line is connected in a daisy-chain fashion.
• INTA signal propagates serially through devices.
• When several devices raise an interrupt-request, INTR line is activated.
• Processor responds by setting INTA line to 1. This signal is received by device 1.
• Device-1 passes signal on to device 2 only if it does not require any service.
• If device-1 has a pending-request for interrupt, the device-1
  → blocks INTA signal &
  → proceeds to put its identifying-code on data-lines.
• Device that is electrically closest to processor has highest priority.
• **Advantage:** It requires fewer wires than the individual connections.

Arrangement of Priority Groups

• Here, the devices are organized in groups & each group is connected at a different priority level.
• Within a group, devices are connected in a daisy chain. (Figure 4.8b).



(a) Daisy chain



(b) Arrangement of priority groups

Figure 4.8 Interrupt priority schemes.

Module 3 :Question Bank

1. Explain the different functional units of a computer.
2. Describe the operational concepts between the processor and memory (or)
3. With a neat diagram, discuss the operational concepts of a computer(or)
4. Draw and explain the connection between memory and processor with the respectiveregisters.
5. Explain the steps to execute an instruction.
6. Explain the bus structure of computer system
7. What is performance measurement? Or How to measure the performance of the computer? Explain the overall SPEC rating for the computer in a program suite
8. Explain the concept of memory location and address.
9. Explain little endian and big-endian memory assignment. Represent the number 64243848H in 32 bits big endian and little-endian memory.
10. Explain the two types of memory operations.
11. Explain INSTRUCTION EXECUTION & STRAIGHT-LINE SEQUENCING.
12. Explain the concept of Branching.
13. Explain conditional codes.
14. What is an interrupt? Explain the transfer of program control through the use of interrupts with an example.
15. Explain interrupt hardware. / Explain how interrupt requests from several IO devices can be communicated to a processor through a single INTR line.
16. Explain the following terms w r. to interrupt
17. Interrupt service routine (ISR) ii) Interrupt latency iii) enabling and disabling of interrupts.
18. Explain in detail, the situations where a number of devices capable of initiating interrupts are connected to the processor? How to resolve the problems? /Explain following methods of handling interrupts from multiple devices: i)Interrupt nesting/priority structure
    ii)Vectored interrupt
    iii)Simultaneous requests/Daisy chain technique.

Module 3 :Tutorial Questions

1) List the steps needed to execute the machine instruction **Add LOCA, R0** in terms of transfer between processor and the memory along with some simple control commands. Assume that the instruction itself is stored in the memory at location INSTR and that thisaddress initially in registers PC.

2) Repeat Problem 1 for the machine instruction **Add R1, R2, R3**

3) Represent the decimal values 4, -4, 13, -13, 25, -22, 30 and -23 as signed 6-bit numbers in the following binary formats:

(a) sign-and-magnitude
(b) 1's-complement
(c)2's-complement

4)Convert the following pairs of decimal numbers to 5-bit 2's-complement numbers, then add them. State whether or not overflow occurs in each case.

a) -5 and 11 b) 5 and -13
c) 14 and -12 d) 6 and -7
e) –4 and –14

5)Repeat Problem 1.7 for the subtract operation, where the second number of each pair is to be subtracted from the first number. State whether or not overflow occurs in each case.

6)Perform the following operations on the 5 bit signed numbers using 2's complement representation system. Also indicate whether overflow has occurred.

(i)(-10)+ (-13) (ii) (-10) - (+4) (iii) (-3) + (-8) (iv) (-10) - (+7)

7)Perform following operations on the 6-bit signed numbers using 2's complement representation system. Also indicate whether overflow has occurred.

i) 111010 + 010011
ii) 100111 – 100010
iii) 101000 + 100100
iv) 010000 - 101000

8) Show the Little endian and Big-endian storage of data for the following datawords

i) 00011100 00001111 01011100 11001100 11001100 11011110 01110100 01010111 10100111 11110111

ii) The string " ComputerOrganization" . Represent each character using its ASCII code

iii) ABCDEF1745AC45FE73452DEA34 (16)

9)Assume the starting memory location addresses is 5000 and the word length of the processor is 32 bits

10)Write the machine instructions using three address, two address, one address, zero address and risc instructions to perform the operation A-B*C.

11) Show the status of Condition Code flags V, N, Z and C after the following operations. Assume the word length of the processor is 8 bits. Assume unsigned numbers.

i) 10111001 + 10001110 ii) 11100011 + 00101011

iii) 11111101 + 10011110 iv) 10100011 + 01101111